

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №2

Выполнил:
Студент группы ИУ5-35Б
Корольков И. А.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Листинг cm_timer.py

```
import time
from contextlib import contextmanager

# Вариант 1 – класс
class cm_timer_1:
    def __enter__(self):
        self.start = time.time()
    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f"time: {time.time() - self.start:.5f}")

# Вариант 2 – contextlib
@contextmanager
def cm_timer_2():
    start = time.time()
    yield
    print(f"time: {time.time() - start:.5f}")

if __name__ == '__main__':
    with cm_timer_1():
        time.sleep(0.5)
    with cm_timer_2():
        time.sleep(0.5)
```

Результат выполнения:

```
koroligor@MacBook-Pro-Igor lab2 % python3 cm_timer.py
time: 0.50504
time: 0.50406
```

Листинг field.py

```
def field(items, *args):
    assert len(args) > 0
    for item in items:
        if len(args) == 1:
            val = item.get(args[0])
            if val is not None:
                yield val
        else:
            d = {k: item.get(k) for k in args if item.get(k) is not None}
            if d:
                yield d

if __name__ == '__main__':
    goods = [
        {'title': 'Ковер', 'price': 2000, 'color': 'green'},
        {'title': 'Диван для отдыха', 'color': 'black'}
    ]
    print(list(field(goods, 'title')))
    print(list(field(goods, 'title', 'price')))
```

Результат выполнения:

```
koroligor@MacBook-Pro-Igor lab2 % python3 field.py
['Ковер', 'Диван для отдыха']
[{'title': 'Ковер', 'price': 2000}, {'title': 'Диван для отдыха'}]
```

Листинг gen_random.py:

```
import random

NUM_COUNT = 5
MIN_VAL = 1
MAX_VAL = 3

def gen_random(num_count, begin, end):
    for _ in range(num_count):
        yield random.randint(begin, end)

if __name__ == '__main__':
    print(list(gen_random(NUM_COUNT, MIN_VAL, MAX_VAL)))
```

Результат выполнения:

```
koroligor@MacBook-Pro-Igor lab2 % python3 gen_random.py
[1, 2, 3, 2, 3]
```

Листинг print_result.py:

```
def print_result(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print(func.__name__)
        if isinstance(result, dict):
            for k, v in result.items():
                print(f"{k} = {v}")
        elif isinstance(result, list):
            for item in result:
                print(item)
        else:
            print(result)
        return result
    return wrapper

@print_result
def test_1(): return 1

@print_result
def test_2(): return 'iu5'

@print_result
```

```

def test_3(): return {'a':1,'b':2}
@print_result
def test_4(): return [1,2]

if __name__ == '__main__':
    test_1()
    test_2()
    test_3()
    test_4()

```

Результат выполнения:

```

koroligor@MacBook-Pro-Igor lab2 % python3 print_result.py
test_1
1
test_2
iu5
test_3
a = 1
b = 2
test_4
1
2

```

Листинг process_data.py:

```

import json
import random
from print_result import print_result
from cm_timer import cm_timer_1
from unique import Unique

path = 'data_light.json'

with open(path, encoding='utf-8') as f:
    data = json.load(f)

@print_result
def f1(arg):
    return sorted(Unique([d['job-name'] for d in arg], ignore_case=True),
key=str.lower)

@print_result
def f2(arg):
    return list(filter(lambda x: x.lower().startswith('программист'), arg))

@print_result
def f3(arg):

```

```

        return list(map(lambda x: f"{x} с опытом Python", arg))

@print_result
def f4(arg):
    salaries = [random.randint(100_000, 200_000) for _ in arg]
    return [f"{job}, зарплата {salary} руб." for job, salary in zip(arg, salaries)]

if __name__ == '__main__':
    with cm_timer_1():
        f4(f3(f2(f1(data))))

```

Результат выполнения:

```

koroligor@MacBook-Pro-Igor lab2 % python3 process_data.py
f1
Аналитик данных
Дизайнер UI/UX
Программист C#
Программист C++
Программист Go
Программист Java
Программист Python
Тестировщик
f2
Программист C#
Программист C++
Программист Go
Программист Java
Программист Python
f3
Программист C# с опытом Python
Программист C++ с опытом Python
Программист Go с опытом Python
Программист Java с опытом Python
Программист Python с опытом Python
f4
Программист C# с опытом Python, зарплата 197109 руб.
Программист C++ с опытом Python, зарплата 136347 руб.
Программист Go с опытом Python, зарплата 143496 руб.
Программист Java с опытом Python, зарплата 101892 руб.
Программист Python с опытом Python, зарплата 151096 руб.
time: 0.00006

```

Листинг sort.py:

```

if __name__ == '__main__':

```

```

user_input = input("Введите числа через пробел: ")

data = [int(x) for x in user_input.split()]

result = sorted(data, key=abs, reverse=True)
print("Сортировка по модулю без lambda:", result)

result_with_lambda = sorted(data, key=lambda x: abs(x), reverse=True)
print("Сортировка по модулю с lambda:", result_with_lambda)

```

Результат выполнения:

```

koroligor@MacBook-Pro-Igor lab2 % python3 sort.py
Введите числа через пробел: 1 2 3 5 8 1
Сортировка по модулю без lambda: [8, 5, 3, 2, 1, 1]
Сортировка по модулю с lambda: [8, 5, 3, 2, 1, 1]

```

Листинг unique.py:

```

class Unique:
    def __init__(self, items, **kwargs):
        self.items = iter(items)
        self.ignore_case = kwargs.get('ignore_case', False)
        self.seen = set()

    def __iter__(self):
        return self

    def __next__(self):
        for item in self.items:
            val = item.lower() if self.ignore_case and isinstance(item, str) else item
            if val not in self.seen:
                self.seen.add(val)
                return item
        raise StopIteration

if __name__ == '__main__':

    user_input = input("Введите элементы через пробел: ")
    data = user_input.split()

    ignore_case_input = input("Игнорировать регистр? (y/n): ").lower()
    ignore_case = ignore_case_input == 'y'

    unique_iter = Unique(data, ignore_case=ignore_case)

    print("Уникальные элементы:")

```

```
for item in unique_iter:  
    print(item)
```

Результат выполнения:

```
koroligor@MacBook-Pro-Igor lab2 % python3 unique.py  
Введите элементы через пробел: 2 3 4 4 5 5 7 7 8 8 8  
Игнорировать регистр? (y/n): y  
Уникальные элементы:  
2  
3  
4  
5  
7  
8
```